Universal Render Pipeline lights and effects in Gem Hunter Match

Make your 2D puzzle game stand out with Unity 2022 LTS

2D puzzle/match 3 games are popular for a few good reasons. They're cute and colorful, easy and fun to play, and accessible to anyone from almost anywhere – on a plane, on the subway to work, or from your living room couch.

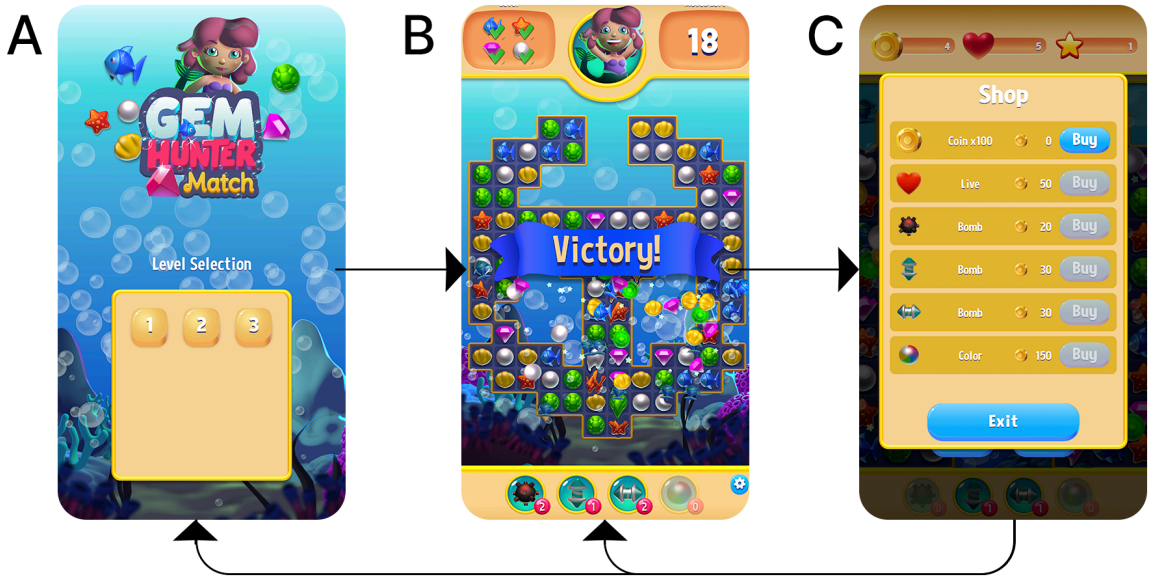
With their static camera and instantly recognizable boards crowded with colorful items, puzzle games aren't famous for bleeding-edge graphics. But developers are raising the quality bar for the genre by using fun storylines and metagame techniques, along with lively animated characters and glimmering visual effects for the board pieces.

With the advancements in mobile graphic capabilities (even in low-end devices) along with the Universal Render Pipeline (URP) in Unity, you can add tons of visual appeal and new gameplay opportunities to your 2D puzzle games.

The main features in Gem Hunter Match

Gem Hunter Match is a playable sample of a match 3 puzzle game designed mainly for mobile, but able to run on multiple platforms. It shows 2D lighting and visual effects created in URP in Unity 2022 LTS. The project organization should be accessible to both programmers and designers. You can customize Gem Hunter Match with your own assets or gameplay. Or, reuse any of its sprites, shaders, effects, audio, textures, and scripts in your project. Please see the Unity Asset Store EULA for additional information.

Sample game loop



The game loop in Gem Hunter Match

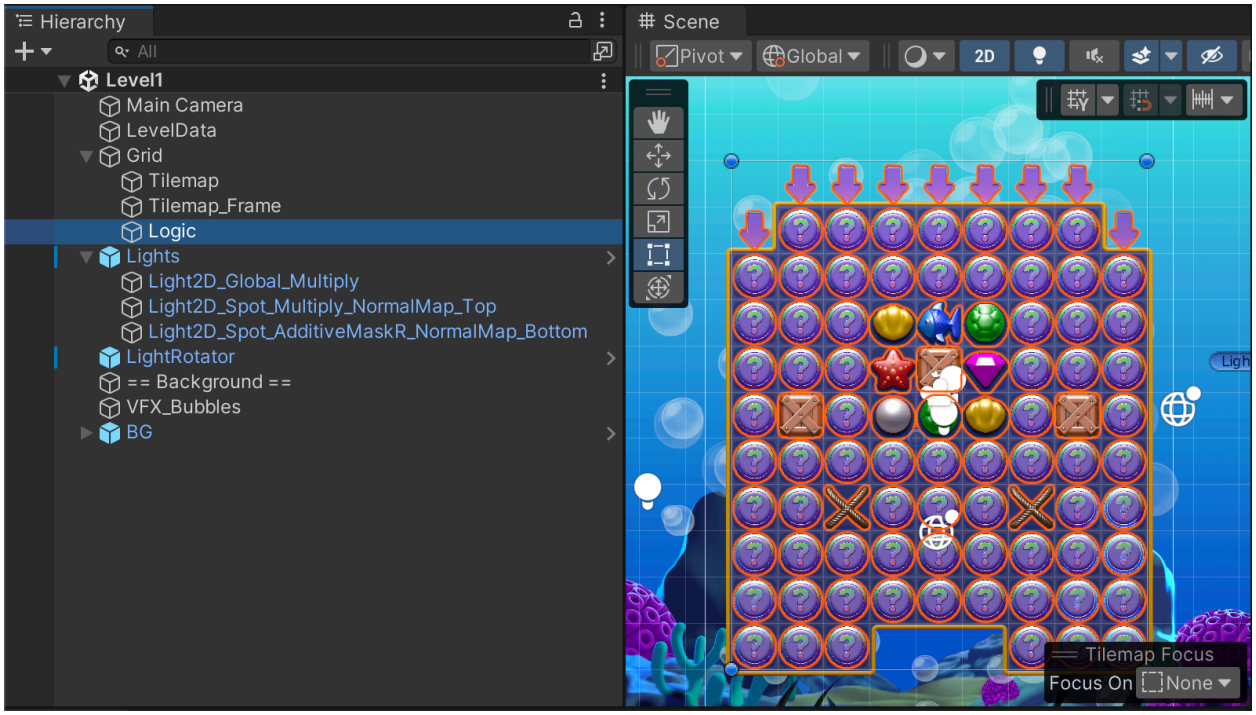
The game loop is simple, with an inventory you can reuse in different ways. Here are its main elements:

A. **The Main scene:** This screen lists all of the playable levels; these are referenced from a ScriptableObject called **LevelList**, located inside the **Data** folder.
B. **The Level scene:** This shows the gameplay setup. You need to clear the elements in the **Goals** section. Clear the gems, and earn boosters and coins, by matching three or more of the same color. The boosters help you achieve the goals; if you fail, you lose a heart. Wooden crates and rope are blockers; match three next to a wooden crate, or underneath a piece of rope, to remove them.

a. In the **Data/BonusItems** folder you'll find the boosters and the referenced prefab that contains parameters like the combo shape to be spawned (in the case of several pieces having the same combo shape, a random one will be chosen). You can create your own boosters from the top menu via **Assets > Create > 2D Match**.

C. **End of level/ The Shop:** Access the shop when you complete, or fail to complete, a level; buy yourself boosters, hearts, or other currency. All of the shop items are in the folder named **Data/ShopItems** (you can also add your own via **Assets > Create > 2D Match**. Items in the shop include:
   a. **Coins:** Earn coins with matches of three or more and use them as soft currency.
   b. **Hearts or lives:** These boosters buy you a chance to retry a failed level. If the player runs out of such boosters in a match 3 game, there's often a cool down period for them before they can replenish their life/health points.
   c. **Stars:** You collect these after completing each level; in actual match 3 games, stars are often a part of the metagame, as decoration, or to advance the storyline.

The flow of a level scene



Level scenes have a simple structure.

Here's a rundown of the main elements in a level scene.

- **The Main Camera:** This is a basic static camera. The script called **LevelData** will adjust its size based on the available screen space and playable area.
- **The LevelData script:** This script contains the goals for the level, number of moves, a background track, and border information for the camera size calculation.
- **The Grid GameObject:** This GameObject contains a script called **Board** that determines the type of gems available in a level, as well as references to VFX Graph-based visual effects.
- **The child GameObjects**: There are a few child GameObjects included in the Grid GameObject. The child GameObjects include:
    - **Tilemap:** This is used to create the background tiles and decoration. The 2D Tilemap system provides visual, out-of-the box level design tools that work especially well for grid-based games. For example, the Tilemap API offers convenient ways to track the position of the many pieces.
    - **Tilemap_Frame:** This is used to create the frame decoration around the background tiles.
    - **Logic:** This contains placeholder tiles for the level design; it's from here that all the placeholder "random gem" tiles (indicated by a question mark) are swapped out for actual gameplay GameObjects (generated randomly via code). However, you can also choose the type of gem or item to spawn on a given tile position, by using the other tiles in the palette.

- **The Lights prefab:** This contains the 2D lights for the grid. The lights in *Gem Hunter Match* affect the default Sprite Lit shader and are applied to the grid items included in the Sorting Layer that receives light. The types of lights in use are:
    - **Light2D_Global_Multiply**: This provides the general lighting and tone applied to the scene; read about the details of each type of 2D light in the documentation.
    - **Light2D_Spot_Multiply_NormalMap_Top**: This is a spot light that affects the normal maps of the elements and highlights the board area.
    - **Light2D_Spot_AdditiveMaskR_NormalMap_Bottom:** This spot light affects the mask map of sprites, resulting in a rim light effect.
- **The LightRotator prefab:** This contains the **LightManager** script that modifies a parameter in the **TileShader** Shader Graph that illuminates the gems with the Sprite Custom Lit shader.
- **VFX_Bubbles** and **BG:** These are decorative background elements; expand the BG GameObject to see how the environment is assembled.

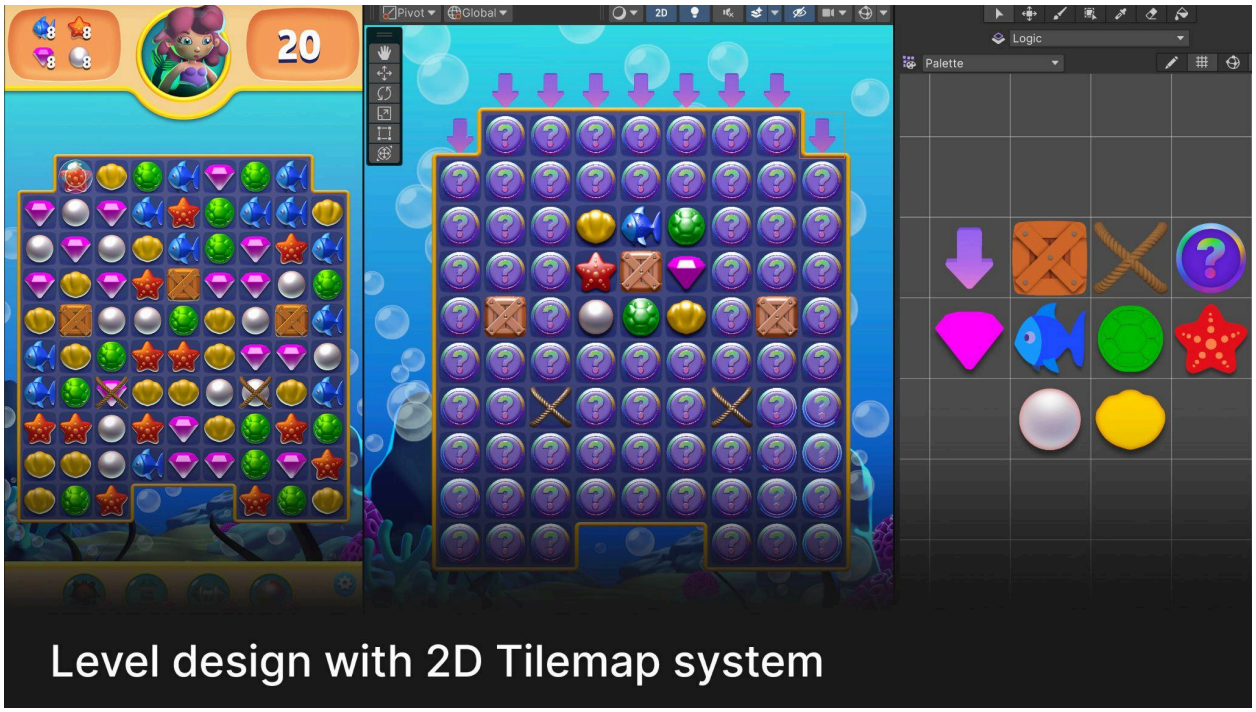
The GameManager prefab

The **GameManager** is the interface between all the systems in the game. It is either instantiated by the loading scene at the start of the game (**Init** scene), or loaded from the **Resource** Folder dynamically when you test a scene directly in the Editor. This allows you to enter Play mode from any point of the game without having to add it to every scene.

```
public class GameManager : MonoBehaviour
{
    //This is set to true when the manager is deleted. This is useful as the manager can be deleted before other
    //objects
    private static bool s_IsShuttingDown = false;

    public static GameManager Instance
    {
        get
        {

            // In Editor, the instance can be crated on the fly so we can play any scene without setup to do.
            // In a build, the first scene will Init all that so we are sure there will already be an instance.
#if UNITY_EDITOR
            if (s_Instance == null && !s_IsShuttingDown)
            {
                var newInstance = Instantiate(Resources.Load<GameManager>("GameManager"));
                newInstance.Awake();
            }
#endif
            return s_Instance;
        }

        private set => s_Instance = value;
    }
```

The GameManager in code

Create your own level



The board gems and other items during gameplay, left, and the placeholder GameObjects, right

You can use the assets in Gem Hunter Match in your projects or add your content to the sample. Go to **File > New Scene** to use the game template.

Follow these steps to make your own level design:

1. Open the **Tile Palette** window
2. Select the Palette; you should see the frame artwork on the left side and the placeholder pieces on the right of the tile palette.
3. In the Hierarchy select the **Grid** GameObject and its child GameObject **Logic**. Hide the Tilemap and Tilemap_Frame GameObjects for convenience while designing the layout.
4. Remove the existing design with the eraser.
5. Paint the layout of the pieces, blockers, and spawning points from where new gems will drop (the arrows).
6. Once you are satisfied with the design, erase and paint the background and frame, selecting and enabling the corresponding tilemaps again.

Remember to modify the **LevelData** GameObject that contains the moves and goals information. And make your level accessible by adding it to the level selection screen in the **LevelList** asset.

Customize the design

An example of how you can customize *Gem Hunter Match* by using the farm-themed assets from *Happy Harvest*, another Unity 2022 LTS 2D URP sample

You can customize the game visuals by replacing sprites of the core playable pieces in the level scenes. The UI can be customized in UI Toolkit's UI Builder. A great resource for learning how to create user interfaces in Unity is this e-book.

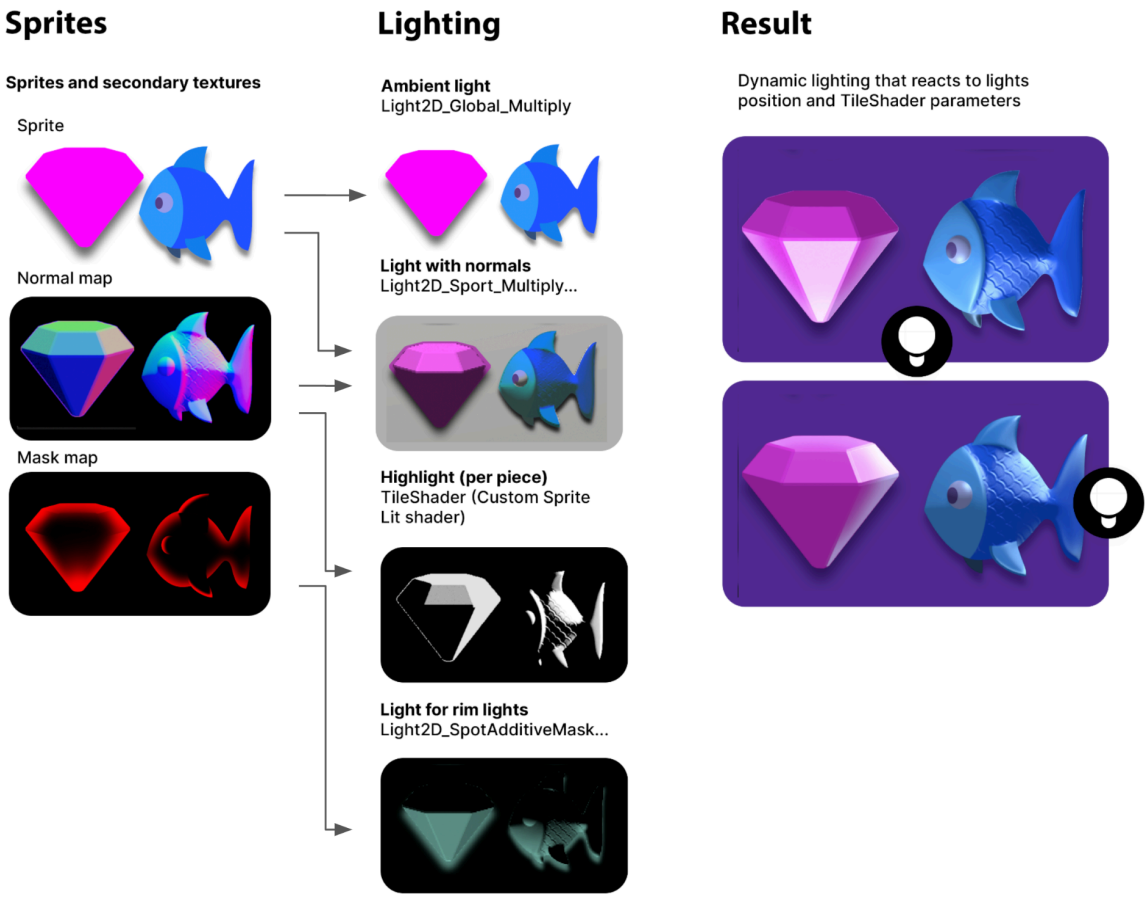
You can also replace the mermaid with a character of your own. We cover the complete workflow to make skeletal animated characters in the e-book 2D game art, animation, and lighting in Unity. Here are a few ideas for how to personalize *Gem Hunter Match*:

- Swap the artwork for each piece from its prefab. If you want to use the sample's dynamic 2D light system, you'll also need to use its methods for your artwork, mainly the decoupling of the shader information system with secondary textures.
    - Inside each prefab you'll find the **UI sprite** reference which is the sprite used in the UI. You can create a new image for the pieces icon; since the sprites will only be used by the UI system, they won't use the 2D lighting system.
- Swap the background for any prefab or image of your own. Disable the **BG** and **VFX_Bubbles** GameObjects so only your new art is visible. Ensure the sprites or your background use the Sorting Layer called **Background** so they don't cover the board.

- You can easily change the frame and background tiles color by selecting Tilemap and Tilemap_frame and changing the color.

Creating the 2D board pieces and custom light

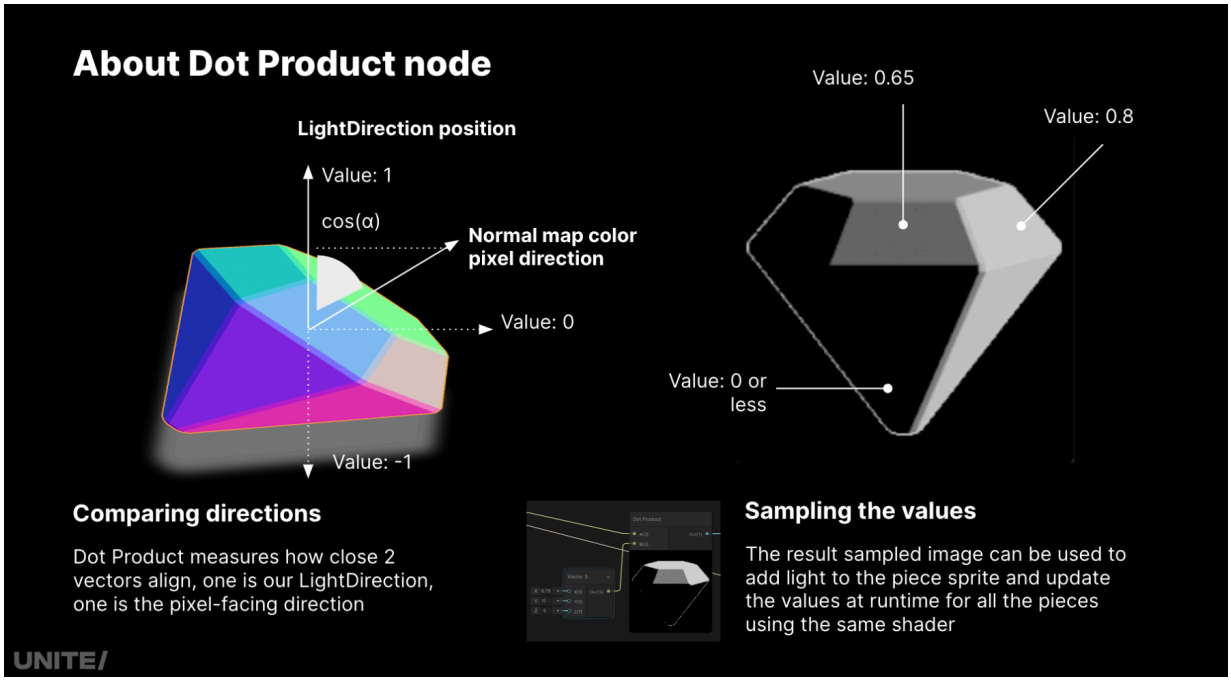
The preparation of your sprites is your key consideration when working with the 2D lighting system. The main sprite should hold color information only, with light and shadow information kept off it, as you'll see in the sample. The normal map is used by the 2D light system to know the direction of each pixel so they receive more or less light based on its position. The mask map is used by lights that can affect a specific RGB channel. This presentation provides a good explanation on the topic.



Sprite preparation: The different lights in the game use the sprite information to produce dynamic 2D lighting

Sometimes due to gameplay, convenience, or art direction needs, you'll want to use a custom lighting model. You can achieve this with a Sprite Custom Lit Shader Graph, which enables you to modify the 2D light texture information, and/or illuminate the sprite in

creative ways. In the sample, for example, a "fictional" light position is represented by the **LightRotator** GameObject, which is animated to create a glare effect off the gems. The modifications to the 2D light texture and the fabricated highlights with the **Dot Product node** are both used in the **TileShader** Shader Graph that's applied to the gems in the game.



Using the Dot Product node in Shader Graph for creating effects on a gem in Gem Hunter Match

The Dot Product node calculates the proximity of the vector coming from the pixel in the normal map to the "fictional" light position. A value of 1 is the closest alignment and can be sampled as a white pixel. You can apply effects like multiplying to make the pixel of the base sprite lighter or darker. The changes in the shader affect all of the elements that use it, such as the puzzle pieces.

Effects with VFX Graph and Shader Graph

The VFX Graph- and Shader Graph-based visual effects in *Gem Hunter Match* include explanatory comments in each of their graphs, to help you understand how each system works together with 2D lights.

One of your best resources for learning how to create visual effects with VFX Graph is the e-book *The definitive guide to creating advanced visual effects in Unity*.

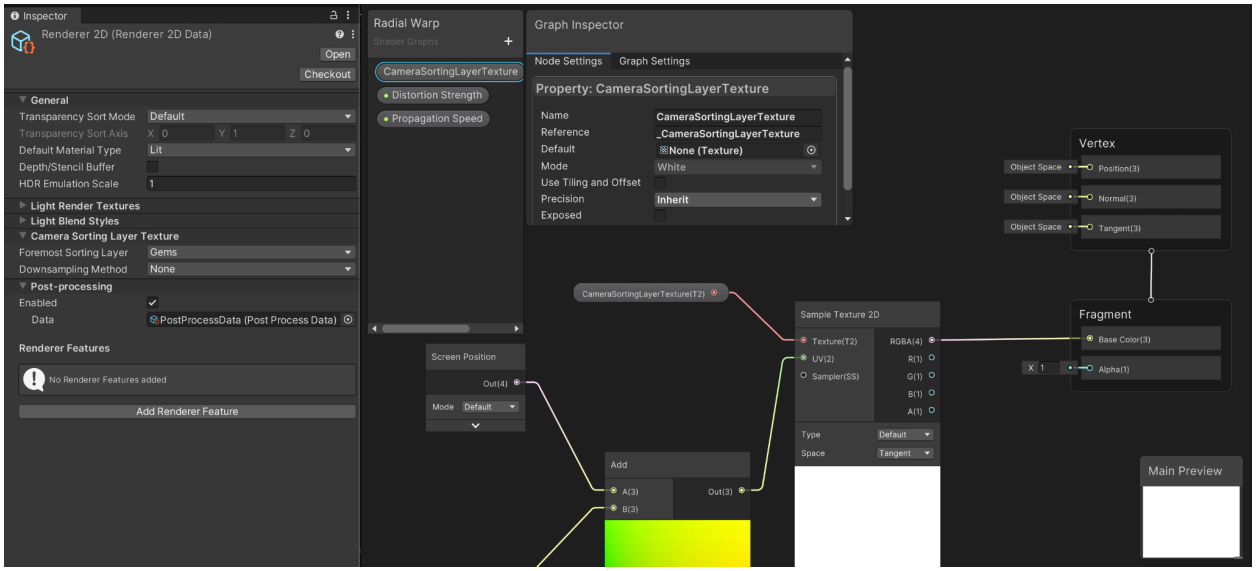
The special effects in Gem Hunter Match include:

- **VFX Graph-based effects that display an unlit particle:** This effect uses the Output Particle Quad.
- **VFX Graph-based particle effects that use a custom shader**: These use a shader graph to render the particles. For example, the effect called **VFX_CellHoldDistortion** uses a 2D shader in the output with the **Support VFX Graph** option enabled in the shader. When you add the 2D Shader Graph in the Output node, the output becomes one of the following:
    - **Output Particle Sprite Unlit Quad:** When the Master node in the Shader Graph is Sprite Unlit, you can still apply effects to the particle but it won't react to 2D Lights.
    - **Output Particle Sprite Lit Quad**: When the Master node in the Shader Graph is Sprite Lit, it will react to 2D lights.

Polished-looking effects are usually a coordinated sequence of effects smoothly linked together with several Spawn nodes that spawn in specific moments with GPU events. When you create your own particle effects, make sure to use the right Sorting Layer in the Inspector for each VFX Graph

Camera Sorting Layer Texture

The **Radial Warp** shader uses the URP 2D Camera Sorting Layer Texture setting. This handy feature gives you access to the graphics generated up to the indicated Sorting Layer in the URP 2D Renderer settings, which you can then use in Shader Graph to apply effects. In the *Happy Harvest* sample the Camera Sorting Layer Texture is used to create a water refraction effect, and in *Dragon Crashers* it's used for smoke distortion. In this sample we use it to apply a distortion that simulates a shockwave, adding extra visual appeal when you make a match.



Caption: The Renderer 2D Data Asset, far left, and the Radial Warp shader read the onscreen image as a texture for the Base Color of the shader, and then apply deformation in the UV

Sharpen your skills with Gem Hunter Match and more Unity resources

We hope you'll take a closer look at other features in this sample, like the 2D animated mermaid character, the way we use some of the UI Toolkit capabilities, and how we implement an object pooling pattern or the audio manager. There is a whole library of technical e-books and related samples that you can download to learn about these methods, toolsets, coding practices, and many more. Be sure to get your copy of these great resources:

- Happy Harvest sample
- Dragon Crashers sample
- QuizU – A UI Toolkit sample
- 2D game art, animation, and lighting in Unity
- Level up your code with game programming patterns
- Create modular architecture with ScriptableObjects
- User interface design and implementation in Unity